Tecnológico de Costa Rica

Campus Central Cartago

Informe - Neo4j

IC430X - Arquitectura e Ingeniería de Datos

Profesor: Keneth Obando Rodríguez

II Semestre - 2026

Estudiantes:

Isaac Jiménez Blanco - 2024202804

Andrés Gabriel Padilla Robles - 2025137295

Alina Rodriguez Wong - 2024063444

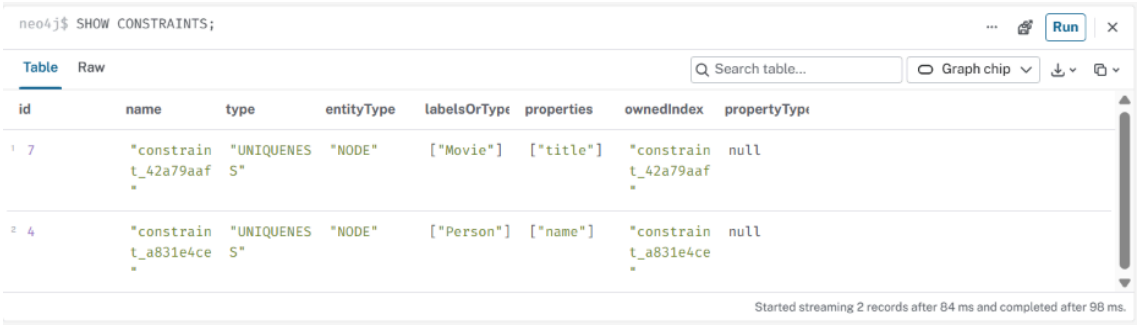
15 de octubre de 2026

1. Descripción breve del ambiente utilizado.

Se utilizó Neo4j 5.26.32 ejecutado mediante un contenedor de Docker en Windows. El contenedor se levantó con la imagen de neo4j:5, poniéndola en el puerto 7474 para la interfaz web, en donde se realizaron las consultas Cypher. Además, se usó el puerto 7687 para el protocolo Bolt para enviar las consultas y recibir los resultados. Los datos se persistieron mediante un volumen de Docker llamado neo4j_data para que la base de datos se mantenga disponible entre reinicios del contenedor.

El profesor Keneth no mando el script de datos para realizar esta práctica, así que se creó un dataset alternativo generado con asistencia de Claude. Este tomo como base el conjunto de datos público de ejemplo “Movie Graph” de Neo4j. Se verificó que cumpliera con el modelo de datos especificado en la sección 3 de las instrucciones de esta práctica, incluyendo nodos Movie y Person, las relaciones ACTED_IN, DIRECTED, PRODUCED, WROTE, REVIEWED y FOLLOWS y los constraints de unicidad sobre Movie.title y Person.name.

2. Evidencia de carga de datos.







La base de datos es un grafo de propiedades. Las capturas anteriores muestran la verificación inicial: SHOW CONSTRAINTS confirma las restricciones de unicidad sobre Movie.title y Person.name; el conteo por etiqueta da 133 nodos Person y 38 nodos Movie y el conteo por tipo de relación da 172 ACTED_IN, 44 DIRECTED, 15 PRODUCED, 10 WROTE, 9 REVIEWED y 3 FOLLOWS. El modelo se compone de:

- **Nodos Movie:** propiedades title (único), released y tagline. Representan las películas.
- **Nodos Person:** propiedades name (único) y born. Una misma persona puede ser actor, director, productor, guionista o usuario que reseña y sigue a otros.
- **ACTED_IN:** (Person)->(Movie) con la propiedad roles, una lista con los personajes interpretados.
- **DIRECTED, PRODUCED y WROTE:** (Person)->(Movie), sin propiedades, para los demás tipos de participación.
- **REVIEWED:** (Person)->(Movie) con las propiedades summary y rating.
- **FOLLOWS:** (Person)->(Person), seguimiento entre usuarios.

Las restricciones de unicidad evitan nodos duplicados y además crean un índice que Neo4j usa para encontrar rápidamente un nodo por título o nombre. En el Q16 el modelo se enriquece con la relación derivada COLLABORATED_WITH entre personas con las propiedades movies y count.

3. Resultados de las consultas Q1 a Q18.

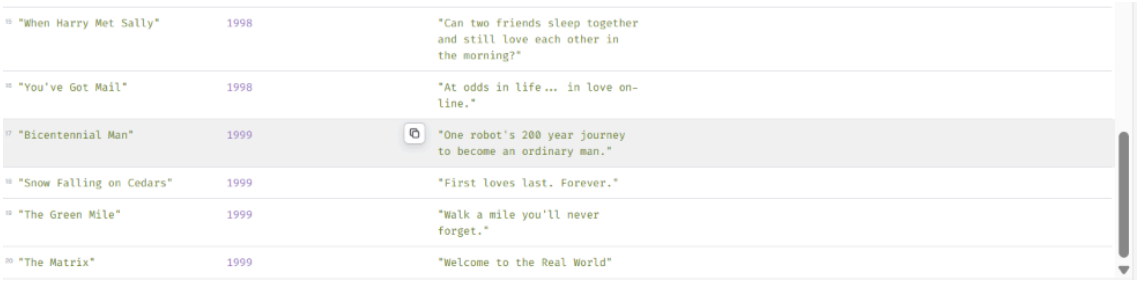
Q1

- Propósito de la consulta: Listar todas las películas del script estrenadas entre 1990 y 1999 con su título, año de estreno y lema de la película, ordenado por año ascendente y luego por título.
- Explicación breve del patrón de grafo utilizado: En esta query se usa el patrón de un único nodo etiquetado, Movie, sin recorrer ninguna relación. Toda la información la tiene el mismo nodo de Movie.
- Captura con el resultado obtenido:





titulo	anio	lema
"Apollo 13"	1995	"Houston, we have a problem."
"Johnny Mnemonic"	1995	"The hottest data on earth. In the coolest head in town"
"That Thing You Do"	1996	"In every life there comes a time when that thing you dream becomes that thing you do"
"The Birdcage"	1996	"Come as you are"
"Twister"	1996	"Don't Breathe. Don't Look Back."
"As Good as It Gets"	1997	"A comedy from the heart that goes for the throat."
"The Devil's Advocate"	1997	"Evil has its winning ways"
"What Dreams May Come"	1998	"After life there is more. The end is just the beginning."



"When Harry Met Sally"	1998	"Can two friends sleep together and still love each other in the morning?"
"You've Got Mail"	1998	"At odds in life... in love on-line."
"Bicentennial Man"	1999	"One robot's 200 year journey to become an ordinary man."
"Snow Falling on Cedars"	1999	"First loves last. Forever."
"The Green Mile"	1999	"Walk a mile you'll never forget."
"The Matrix"	1999	"Welcome to the Real World"

- Comentario sobre la utilidad de la consulta en un sistema real: Esto es útil en sistemas reales de streaming de películas como Netflix, en donde el usuario puede llegar a solicitar una filtración de películas por años o de cualquier otra característica.

Q2

- Propósito de la consulta: Lista todos los actores y los roles que interpretaron en la película The Matrix mostrando el nombre de la persona y la lista de roles.
- Explicación breve del patrón de grafo utilizado: Aca se hace una relación de grafo. Se va de nodos Person hacia un nodo Movie con la relación ACTED_IN. Este nodo Movie es específico ya que se filtra la película que se requiere para este nodo. La variable r captura la relación para acceder a sus roles.
- Captura o tabla con el resultado obtenido:



- Comentario sobre la utilidad de la consulta en un sistema real: Esta consulta sirve para mostrar el patrón base del reparto completo de una película en plataformas reales de streaming.

Q3

- Propósito de la consulta: Lista todas las películas junto con sus directores. Si una película tiene más de un director, los agrupa en una lista.
- Explicación breve del patrón de grafo utilizado: Este patrón recorre la relación de DIRECTED del nodo Person a Movie, empieza en la película y luego va hacia sus directores. Acá se usa la función de agregación collect() para cuando varias personas comparten el mismo valor de m.title, así que Cypher las junta para mostrarlas en una sola lista por película.
- Captura o tabla con el resultado obtenido:

Table Raw

Search table...

	pelicula	directores
1	"A Few Good Men"	["Rob Reiner"]
2	"A League of Their Own"	["Penny Marshall"]
3	"Apollo 13"	["Ron Howard"]
4	"As Good as It Gets"	["James L. Brooks"]
5	"Bicentennial Man"	["Chris Columbus"]
6	"Cast Away"	["Robert Zemeckis"]
7	"Charlie Wilson's War"	["Mike Nichols"]
8	"Cloud Atlas"	["Tom Tykwer", "Lana Wachowski", "Lilly Wachowski"]
9	"Frost/Nixon"	["Ron Howard"]
10	"Hoffa"	["Danny DeVito"]
11	"Jerry Maguire"	["Cameron Crowe"]
12	"Joe Versus the Volcano"	["John Patrick Stanley"]
13	"Johnny Mnemonic"	["Robert Longo"]
14	"Ninja Assassin"	["James Marshall"]
15	"One Flew Over the Cuckoo's Nest"	["Milos Forman"]
16	"RescueDawn"	["Werner Herzog"]
17	"Sleepless in Seattle"	["Nora Ephron"]
18	"Snow Falling on Cedars"	["Scott Hicks"]
19	"Something's Gotta Give"	["Nancy Meyers"]
20	"Speed Racer"	["Lana Wachowski", "Lilly Wachowski"]
21	"Stand By Me"	["Rob Reiner"]
22	"That Thing You Do"	["Tom Hanks"]



23	"The Birdcage"	["Mike Nichols"]
24	"The Da Vinci Code"	["Ron Howard"]
25	"The Devil's Advocate"	["Taylor Hackford"]
26	"The Green Mile"	["Frank Darabont"]
27	"The Matrix"	["Lana Wachowski", "Lilly Wachowski"]
28	"The Matrix Reloaded"	["Lana Wachowski", "Lilly Wachowski"]
29	"The Matrix Revolutions"	["Lana Wachowski", "Lilly Wachowski"]
30	"The Polar Express"	["Robert Zemeckis"]
31	"The Replacements"	["Howard Deutch"]
32	"Top Gun"	["Tony Scott"]
33	"Twister"	["Jan de Bont"]
34	"Unforgiven"	["Clint Eastwood"]
35	"V for Vendetta"	["James Marshall"]
36	"What Dreams May Come"	["Vincent Ward"]
37	"When Harry Met Sally"	["Rob Reiner"]
38	"You've Got Mail"	["Nora Ephron"]

- Comentario sobre la utilidad de la consulta en un sistema real: Para reportes y catálogos se suele utilizar este patrón, ya que agrupa relaciones múltiples en una sola fila. Por ejemplo, mostrar todos los directores de una obra sin duplicar filas por cada persona involucrada en el equipo.

Q4

- Propósito de la consulta: Lista las personas nacidas después de 1965 que hayan actuado en al menos una película, mostrando el nombre, año de nacimiento y cantidad de películas en las que actuaron.
- Explicación breve del patrón de grafo utilizado: Este patrón tiene el mismo recorrido que Q2 solo que no tiene filtro por película, ya que se buscan todas las combinaciones de personas y películas conectadas por ACTED_IN. Sin embargo, sí lleva filtro para verificar que fueron nacidas después de 1965.
- Captura o tabla con el resultado obtenido:

```
neo4j$ MATCH (p:Person)-[:ACTED_IN]->(m:Movie) WHERE p.born > 1965 RETURN p.n
```

Table Raw

	nombre	anio_nacimiento	cantidad_peliculas
1	"Cuba Gooding Jr."	1968	4
2	"Carrie-Anne Moss"	1967	3
3	"Ben Miles"	1967	3
4	"Kiefer Sutherland"	1966	2
5	"Steve Zahn"	1967	2
6	"Philip Seymour Hoffman"	1967	2
7	"Sam Rockwell"	1968	2
8	"Rick Yune"	1971	2
9	"Jerry O'Connell"	1974	2
10	"Charlize Theron"	1975	2
11	"Rain"	1982	2

Table Raw

	nombre	anio_nacimiento	cantidad_peliculas
12	"John Cusack"	1966	1
13	"Halle Berry"	1966	1
14	"Matthew Fox"	1966	1
15	"James Marshall"	1967	1
16	"Julia Roberts"	1967	1
17	"Parker Posey"	1968	1
18	"Orlando Jones"	1968	1
19	"Dina Meyer"	1968	1
20	"Renee Zellweger"	1969	1
21	"Michael Sheen"	1969	1
22	"Jay Mohr"	1970	1

23 "River Phoenix"	1970	1
24 "Ethan Hawke"	1970	1
25 "Brooke Langton"	1970	1
26 "Noah Wyle"	1971	1
27 "Regina King"	1971	1
28 "Corey Feldman"	1971	1
29 "Paul Bettany"	1971	1
30 "Wil Wheaton"	1972	1
31 "Dave Chappelle"	1973	1
32 "Christian Bale"	1974	1
33 "Audrey Tautou"	1976	1

34 "Liv Tyler"	1977	1
35 "Emil Eifrem"	1978	1
36 "Christina Ricci"	1980	1
37 "Natalie Portman"	1981	1
38 "Emile Hirsch"	1985	1
39 "Jonathan Lipnicki"	1996	1

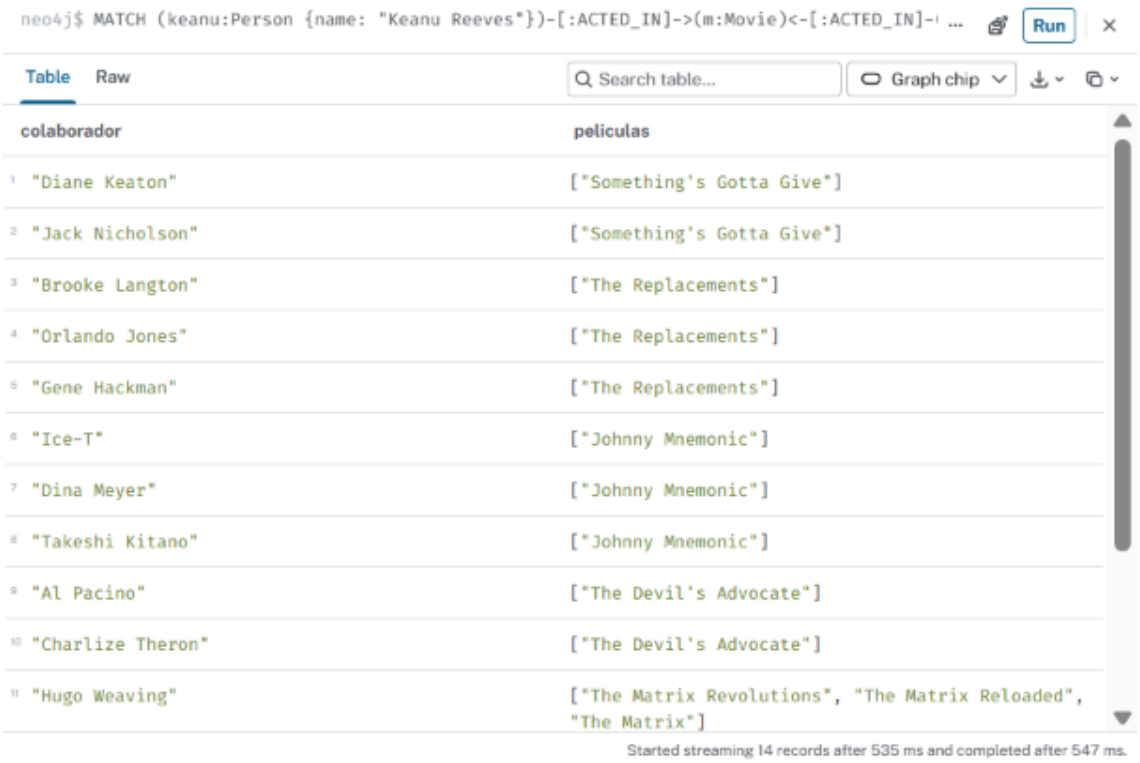
- Comentario sobre la utilidad de la consulta en un sistema real: Este patrón se puede utilizar en sistemas reales para hacer un análisis de características entre las personas. En este caso, para identificar a actores de una generación en especifico.

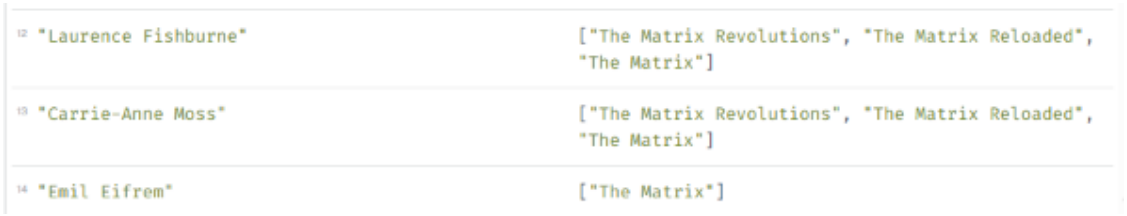
Q5

- Propósito de la consulta: Encontrar todas las personas que han actuado en una misma película con Keanu Reeves, mostrando el nombre del colaborador y las películas en las que colaboraron.
- Explicación breve del patrón de grafo utilizado: Este es un patrón de dos saltos en forma de “V”. Se navega desde el actor Keanu hacia cada película en la que actuó y desde esa misma película se regresa hacia cualquier otra persona que también haya

actuado ahí. El WHERE colaborador <> keanu se usa para que Keanu no salga como su mismo colaborador. Luego, se agrupan las películas que comparten por medio de un collect().

- Captura o tabla con el resultado obtenido:





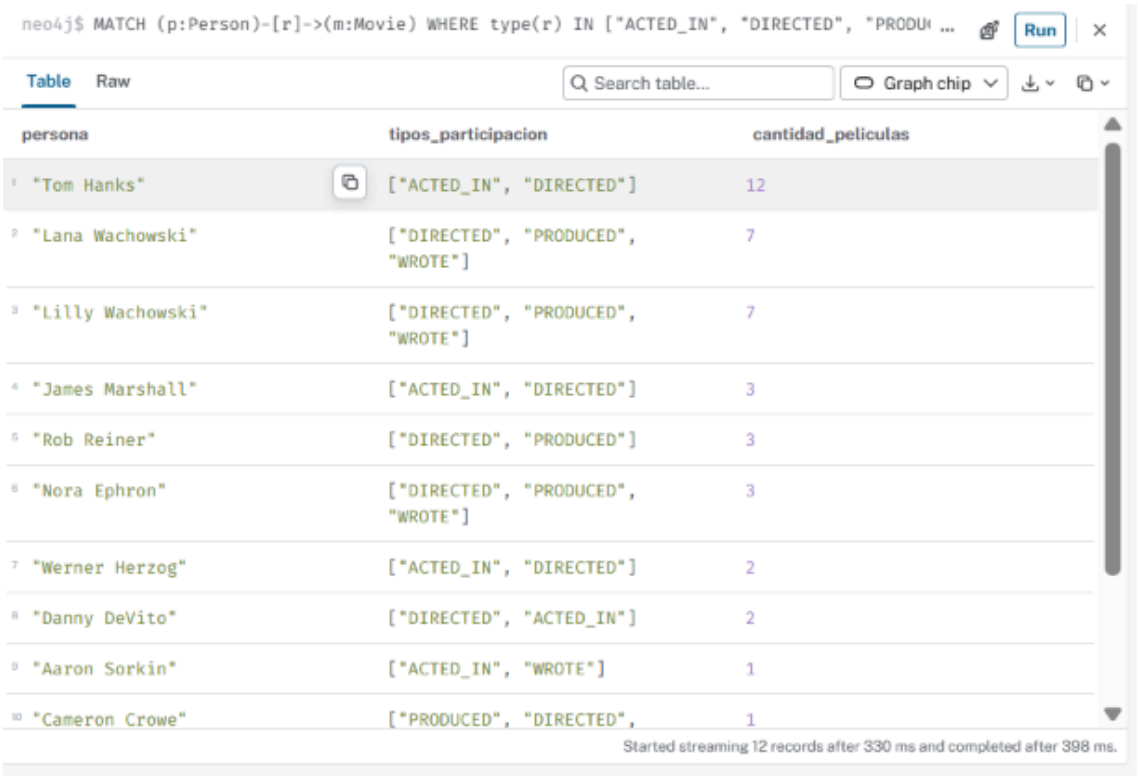
- Comentario sobre la utilidad de la consulta en un sistema real: Esto sirve en sistemas reales para encontrar redes de colaboración profesional en la misma industria.

Q6

- Propósito de la consulta: Encuentra personas que hayan participado en películas con al menos dos tipos diferentes de relación entre ACTED_IN, DIRECTED,

PRODUCED y WROTE, mostrando el nombre de la persona, los tipos de participación y la cantidad de películas relacionadas.

- Explicación breve del patrón de grafo utilizado: En este patrón se usa la variable r para la relación, pero esta no tiene tipo. Type(r) extrae el tipo de la relación mientras se ejecuta, lo que permite ver relaciones de distintos tipos en una sola pasada. El WITH agrupa los resultados por persona antes de continuar para listar los distintos tipos de relación de una persona. El segundo WHERE filtra sobre el resultado ya agregado, algo que no se podía hacer antes, porque size(tipos_participacion) todavía no existía.
- Captura o tabla con el resultado obtenido:



- Comentario sobre la utilidad de la consulta en un sistema real: Esto es útil en sistemas reales para identificar cuando una persona ha tenido varias funciones en una misma industria, que es información valiosa para la carrera de las personas.

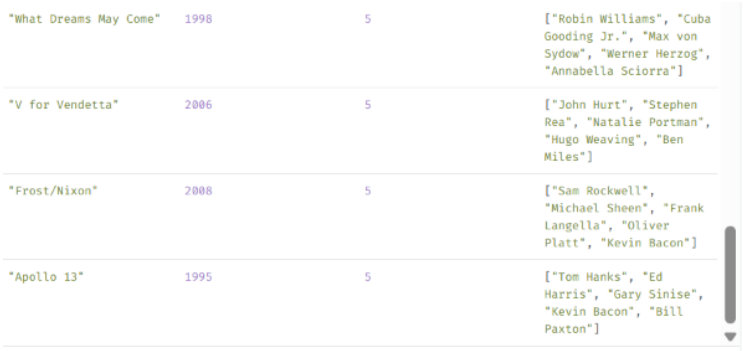
Q7

- Propósito de la consulta: Lista las películas que tengan cinco o más actores registrados, mostrando el título de la película, año, cantidad de actores y lista de actores.
- Explicación breve del patrón de grafo utilizado: En este patrón se recorre ACTED_IN desde Movie hacia Person. El WITH agrupa por película para calcular las agregaciones de la lista de nombres y cuántos actores tiene esa película. El WHERE filtra las películas que alcanzan mínimo 5 actores.
- Captura o tabla con el resultado obtenido:





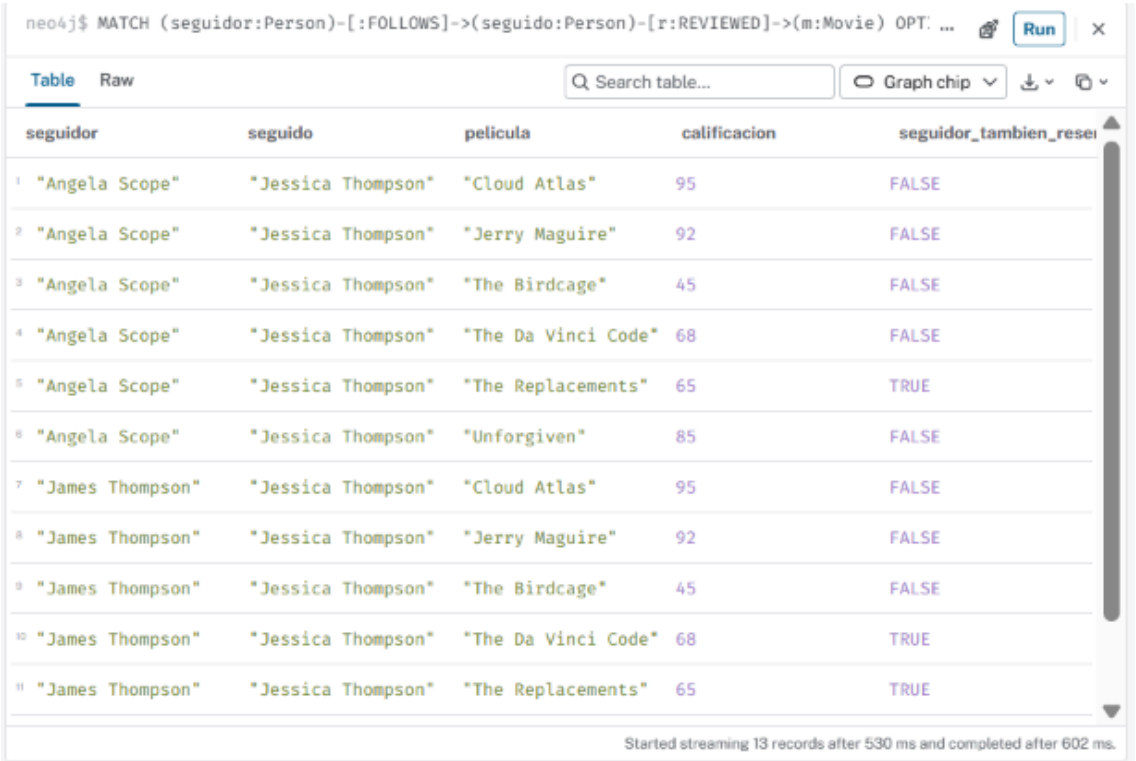




- Comentario sobre la utilidad de la consulta en un sistema real: En un sistema real sirve para identificar qué producciones tienen un grupo de trabajadores grandes para analizar el presupuesto, logística de producción, etc.

Q8

- Propósito de la consulta: Listar las películas reseñadas por las personas seguidas usando las relaciones FOLLOWS y REVIEWED. Muestra quién sigue, a quién sigue, la película reseñada, la calificación y si la persona seguidora también reseñó esa misma película.
- Explicación breve del patrón de grafo utilizado: En este patrón, el primer MATCH encadena dos relaciones distintas. El FOLLOWS que es entre dos personas y el REVIEWED que es de la persona seguida hacia una película. Cypher permite pegar relaciones de tipos diferentes en una sola cadena siempre que los nodos intermedios coincidan. El OPTIONAL MATCH intenta encontrar si el seguidor también reseñó esa misma película, pero si no existe esa reseña, simplemente deja r2 en null. Por eso la expresión final convierte eso en un boolean, verdadero si el seguidor también reseñó, falso si no.
- Captura o tabla con el resultado obtenido:



seguidor	seguido	pelicula	calificacion	seguidor_tambien_rese
"Angela Scope"	"Jessica Thompson"	"Cloud Atlas"	95	FALSE
"Angela Scope"	"Jessica Thompson"	"Jerry Maguire"	92	FALSE
"Angela Scope"	"Jessica Thompson"	"The Birdcage"	45	FALSE
"Angela Scope"	"Jessica Thompson"	"The Da Vinci Code"	68	FALSE
"Angela Scope"	"Jessica Thompson"	"The Replacements"	65	TRUE
"Angela Scope"	"Jessica Thompson"	"Unforgiven"	85	FALSE
"James Thompson"	"Jessica Thompson"	"Cloud Atlas"	95	FALSE
"James Thompson"	"Jessica Thompson"	"Jerry Maguire"	92	FALSE
"James Thompson"	"Jessica Thompson"	"The Birdcage"	45	FALSE
"James Thompson"	"Jessica Thompson"	"The Da Vinci Code"	68	TRUE
"James Thompson"	"Jessica Thompson"	"The Replacements"	65	TRUE

12	"James Thompson"	"Jessica Thompson"	"Unforgiven"	85	FALSE
13	"Paul Blythe"	"Angela Scope"	"The Replacements"	62	FALSE

- Comentario sobre la utilidad de la consulta en un sistema real: Este patrón se utiliza mucho en los sistemas de redes sociales para ver a que han reaccionado, compartido, comentado tus amigos en la plataforma.

Q9

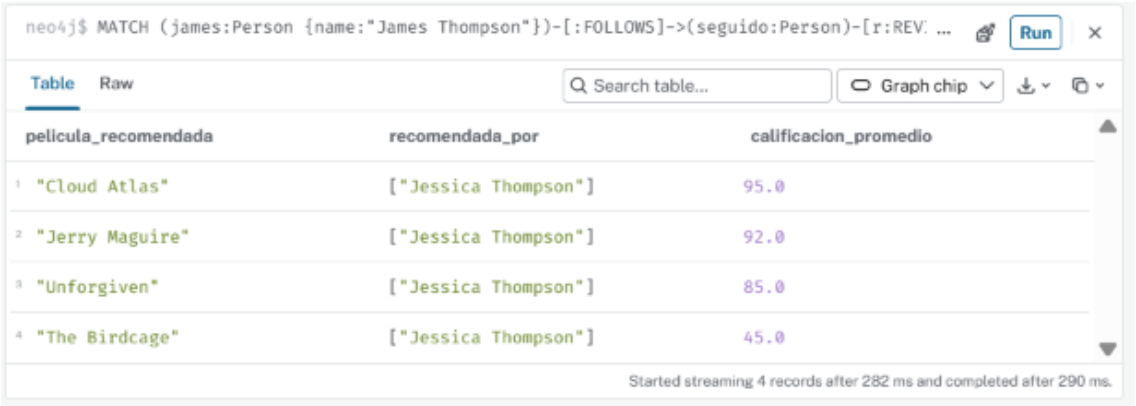
- Propósito de la consulta: Encuentra la ruta más corta de conexión entre Keanu Reeves y Tom Hanks dentro del grafo de participación en películas.
- Explicación breve del patrón de grafo utilizado: Este patrón usa la función shortestPath(), encuentra el camino más corto entre los dos nodos usando un algoritmo tipo BFS en el recorrido de profundidad variable. En este patrón se acepta cualquiera de los cuatro tipos de relación (ACTED_IN | DIRECTED | PRODUCED | WROTE) al recorrer el camino. El resultado p es un objeto de tipo path, nodes(p) extrae la lista de nodos que la componen y la función coalesce(n.title, n.name) devuelve el título si el nodo es una película o el nombre si es una persona. length(p) cuenta cuántos saltos (relaciones) tiene la ruta.
- Captura o tabla con el resultado obtenido:



- Comentario sobre la utilidad de la consulta en un sistema real: Esto se usa en sistemas reales para ver la cercanía entre nodos en una red. Los nodos podrían ser, por ejemplo, personas en LinkedIn y se puede ver que tanta conexión hay entre ellas. Otro ejemplo son los seguidores en común en redes sociales.

Q10

- Propósito de la consulta: Genera recomendaciones de películas para James Thompson basadas en las reseñas de las personas que sigue, excluyendo las que él ya reseñó.
- Explicación breve del patrón de grafo utilizado: Es un patrón de cadena de dos relaciones con la función NOT EXISTS que se usa como filtro de exclusión, descarta del resultado cualquier película que James ya haya hecho reseña sin un MATCH o WHERE NULL. El avg(r.rating) agrega el promedio de calificación cuando varias personas seguidas hicieron reseña de la misma película.
- Captura o tabla con el resultado obtenido:

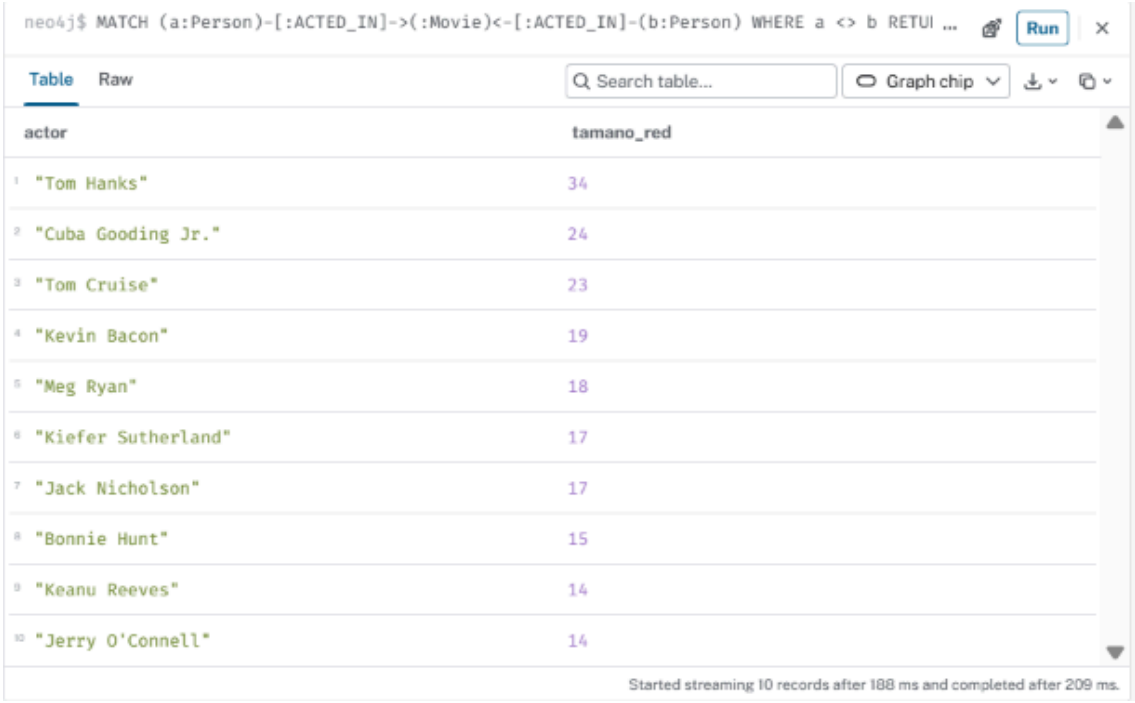


- Comentario sobre la utilidad de la consulta en un sistema real: Este patrón se suele usar en sistemas reales de recomendación colaborativa como los que se usan en redes sociales y otras plataformas para indicar que otra gente que sigues le gusta esto.

Q11

- Propósito de la consulta: Calcula, para cada actor, con cuántos actores distintos ha compartido reparto y muestra el top 10.
- Explicación breve del patrón de grafo utilizado: Usa el mismo patrón de V que en el Q5 (actor1 -> película <- actor2) solo que en vez de fijar a una persona específica, lo aplica sobre todos los actores a la vez. El DISTINCT funciona para que cada colaborador cuente una sola vez sin importar cuántas películas compartan.

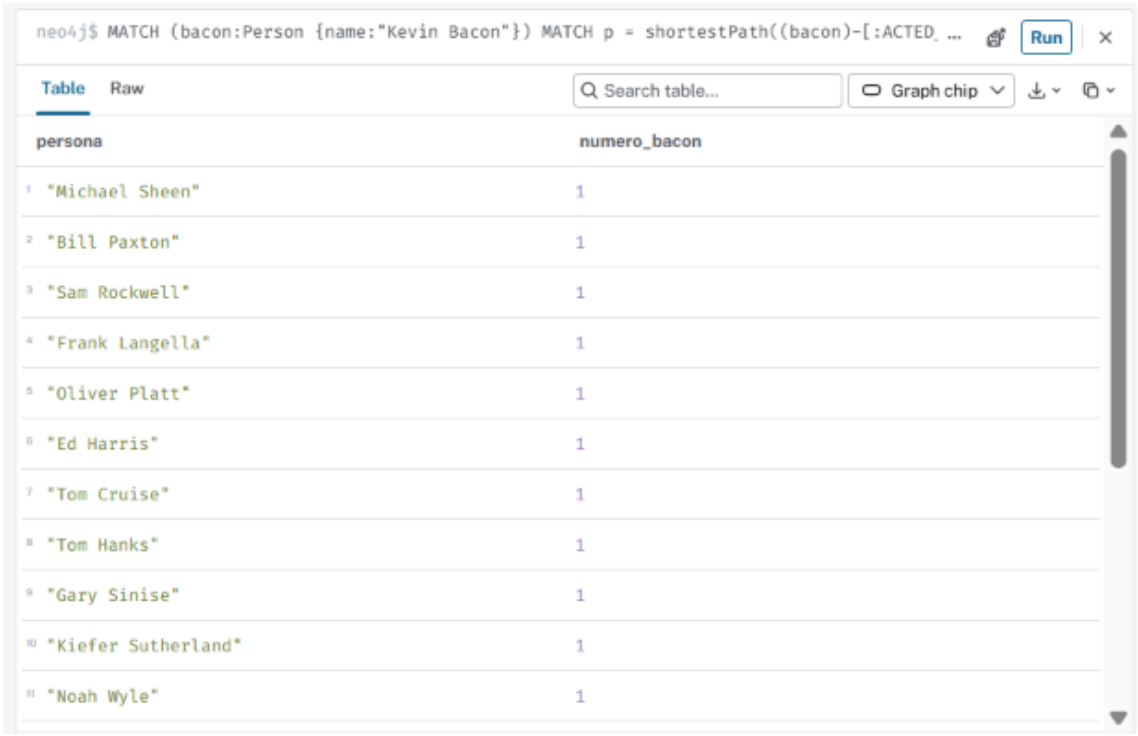
- Captura o tabla con el resultado obtenido:



- Comentario sobre la utilidad de la consulta en un sistema real: Mide la influencia de un nodo dentro de una red social o profesional. Esto es útil para identificar personas clave que conectan distintos grupos dentro de una industria.

Q12

- Propósito de la consulta: Calcula una versión simplificada del “número de Bacon” para las 20 personas más cercanas a Kevin Bacon, midiendo la distancia en cantidad de películas.
- Explicación breve del patrón de grafo utilizado: ACTED_IN solo conecta Person con Movie en parejitas, por eso length(p), que cuenta relaciones y no películas, siempre es el doble de la cantidad real de películas que separan a las dos personas. Esta es la razón para dividir entre 2 (length(p)/2), para obtener el número de Bacon real. Es como el patrón que utiliza shortestpath pero aplicado a todas las personas del grafo de una vez.
- Captura o tabla con el resultado obtenido:

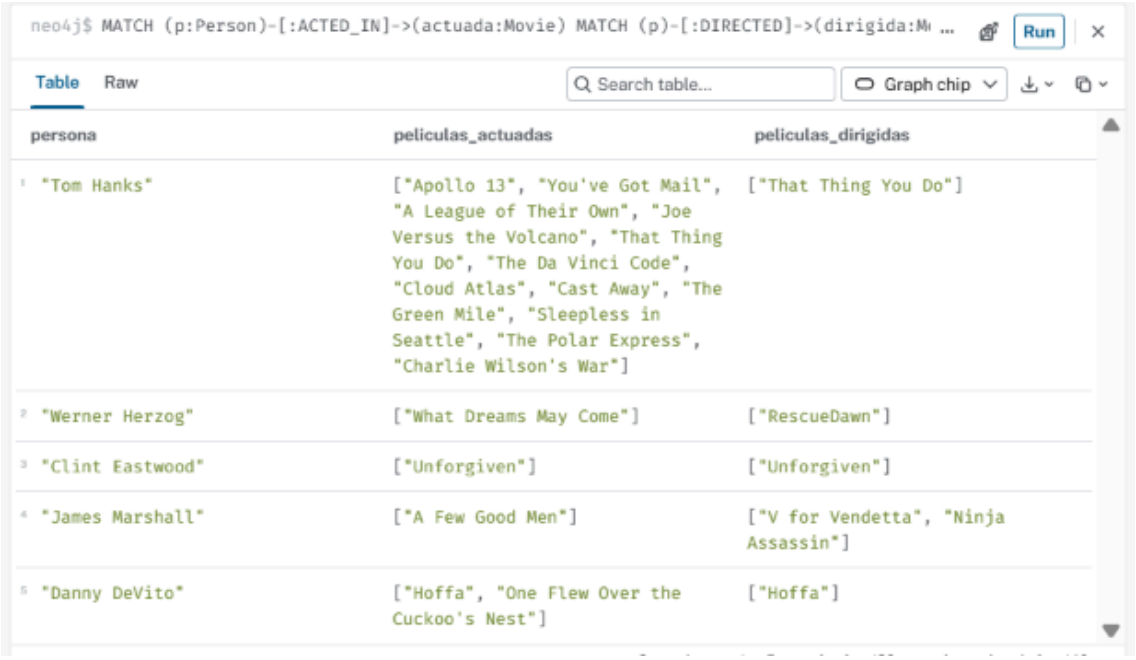




- Comentario sobre la utilidad de la consulta en un sistema real: Esto sirve en sistemas reales para medir el grado de separación entre cualquier par de profesionales dentro de una red de colaboración.

Q13

- Propósito de la consulta: Encuentra personas que tienen los roles de actuar y dirigir en su carrera sin exigir que haya sido en la misma película, listando ambos conjuntos de películas por separado.
- Explicación breve del patrón de grafo utilizado: Se usan dos MATCH independientes sobre la misma variable p en vez de un solo patrón combinado, lo que permite que las películas actuadas y dirigidas sean distintas. Al separarlos en dos MATCH, Cypher los trata como condiciones independientes que deben cumplirse sobre la misma persona, pero permite diferenciar entre los nodos de actuado y dirección.
- Captura o tabla con el resultado obtenido:



	persona	peliculas_actuadas	peliculas_dirigidas
1	"Tom Hanks"	["Apollo 13", "You've Got Mail", "A League of Their Own", "Joe Versus the Volcano", "That Thing You Do", "The Da Vinci Code", "Cloud Atlas", "Cast Away", "The Green Mile", "Sleepless in Seattle", "The Polar Express", "Charlie Wilson's War"]	["That Thing You Do"]
2	"Werner Herzog"	["What Dreams May Come"]	["RescueDawn"]
3	"Clint Eastwood"	["Unforgiven"]	["Unforgiven"]
4	"James Marshall"	["A Few Good Men"]	["V for Vendetta", "Ninja Assassin"]
5	"Danny DeVito"	["Hoffa", "One Flew Over the Cuckoo's Nest"]	["Hoffa"]

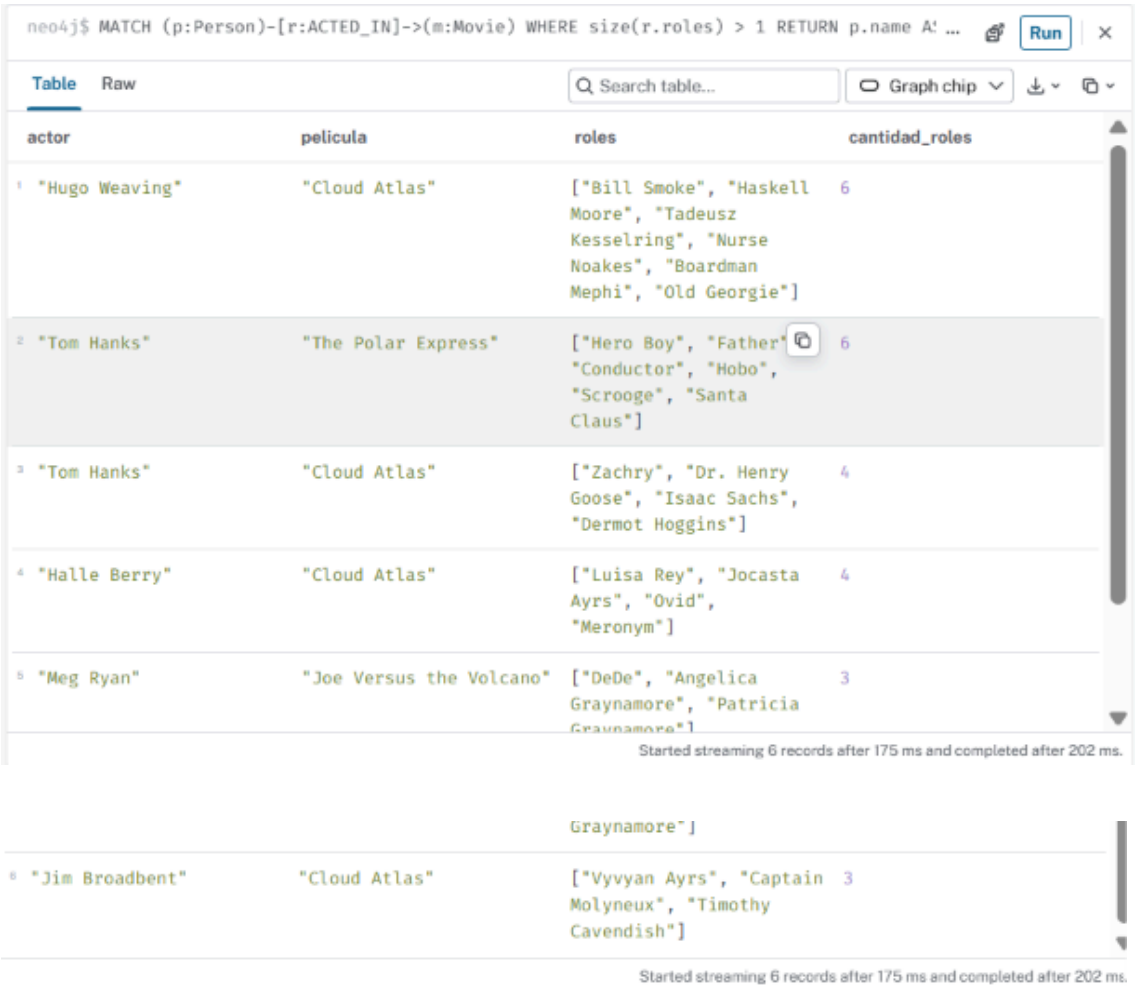
- Comentario sobre la utilidad de la consulta en un sistema real: Esto sirve en el sistema real para identificar personas que puedan hacer dos funciones dentro de la misma industria, que es información útil para analizar carreras profesionales.

Q14

- Propósito de la consulta: Encuentra actores que interpretaron más de un personaje dentro de la misma película.
- Explicación breve del patrón de grafo utilizado: Este patrón es un recorrido de un solo salto, sin agregación. Lo importante es que el dato está en la relación y no en los

nodos. r.roles es una lista de personajes y size(r.roles) > 1 deja solo las relaciones en donde la lista hay más de un elemento. Esto muestra una ventaja del modelo de grafo con propiedades en las relaciones porque el rol pertenece a la participación de la persona en esa película específica.

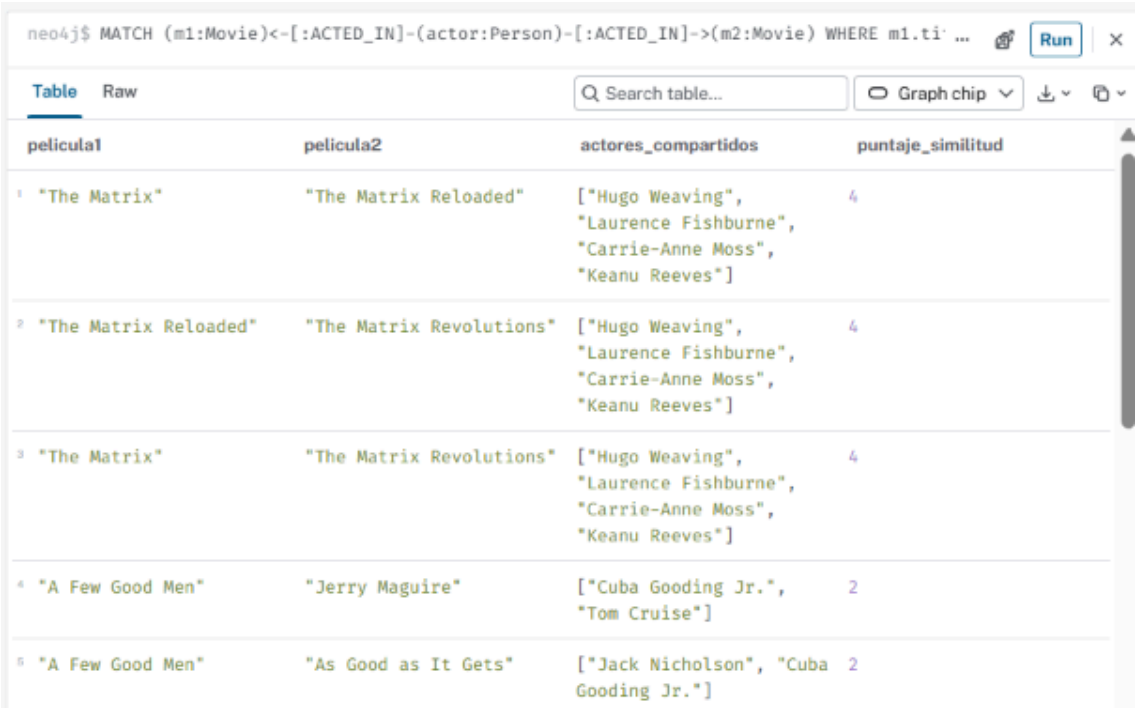
- Captura o tabla con el resultado obtenido:



- Comentario sobre la utilidad de la consulta en un sistema real: Este patrón en sistemas reales sirve para detectar casos especiales en catálogos de reparto, como actores con papeles múltiples que deben aparecer en los créditos y también para validar la calidad de los datos, por ejemplo roles repetidos.

Q15

- Propósito de la consulta: Encuentra pares de películas que comparten al menos dos actores, calculando un puntaje de similitud basado en esa cantidad compartida.
- Explicación breve del patrón de grafo utilizado: Usa el patrón “V” con Movie en los extremos y Person en el centro. La condición WHERE m1.title < m2.title es la parte más importante para que cada par de películas no aparezca dos veces. Al comparar los títulos alfabéticamente y exigir que uno sea estrictamente menor que el otro cada par de películas distintas aparece exactamente una vez. El WITH agrupa por par de películas para contar actores compartidos
- Captura o tabla con el resultado obtenido:



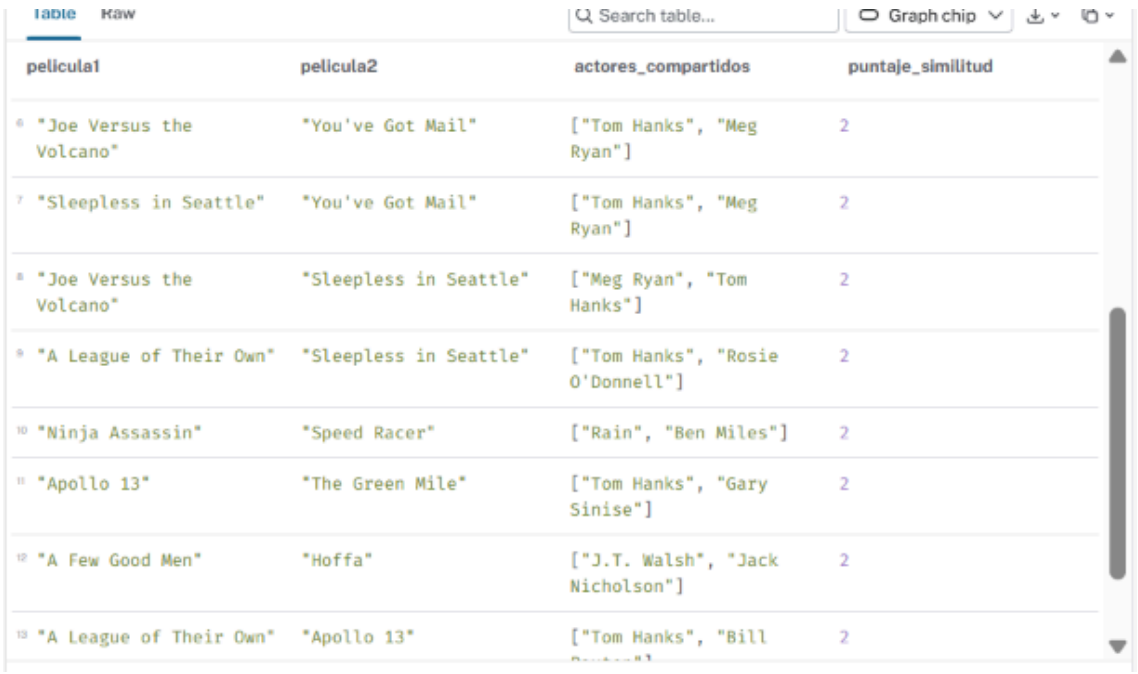
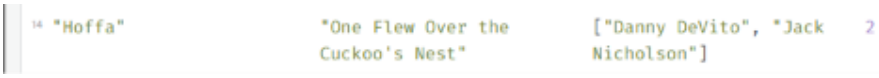


Table Raw | Search table... | Graph chip

	pelicula1	pelicula2	actores_compartidos	puntaje_similitud
6	"Joe Versus the Volcano"	"You've Got Mail"	["Tom Hanks", "Meg Ryan"]	2
7	"Sleepless in Seattle"	"You've Got Mail"	["Tom Hanks", "Meg Ryan"]	2
8	"Joe Versus the Volcano"	"Sleepless in Seattle"	["Meg Ryan", "Tom Hanks"]	2
9	"A League of Their Own"	"Sleepless in Seattle"	["Tom Hanks", "Rosie O'Donnell"]	2
10	"Ninja Assassin"	"Speed Racer"	["Rain", "Ben Miles"]	2
11	"Apollo 13"	"The Green Mile"	["Tom Hanks", "Gary Sinise"]	2
12	"A Few Good Men"	"Hoffa"	["J.T. Walsh", "Jack Nicholson"]	2
13	"A League of Their Own"	"Apollo 13"	["Tom Hanks", "Bill	2



14	"Hoffa"	"One Flew Over the Cuckoo's Nest"	["Danny DeVito", "Jack Nicholson"]	2

- Comentario sobre la utilidad de la consulta en un sistema real: En un sistema real, esta consulta sirve para recomendar películas basado en los gustos que ha tenido la persona previamente, las películas que comparten reparto, etc.

Q16

- Propósito de la consulta: Se conecta el quién colaboró con quién directamente en el grafo, creando una nueva relación COLLABORATED_WITH entre cada par de actores que compartieron al menos una película con el detalle de cuáles películas compartieron y cuántas.
- Explicación breve del patrón de grafo utilizado: Usa el patrón en "V" (a)-[:ACTED_IN]->(m)<-[:ACTED_IN]-(b), pero para escribir en el grafo. La condición a.name < b.name deja solo una dirección por cada par de personas, así que no se crean las relaciones A-B y B-A. El WITH agrupa por par para obtener collect(m.title) y count(m). MERGE crea la relación solo si todavía no existe, por lo que ejecutar la consulta otra vez no genera duplicados y SET guarda las propiedades

movies y count. Se realizaron cambios en el grafo. Se crearon 362 relaciones :COLLABORATED_WITH y se establecieron 724 propiedades (2 por relación: movies y count). No se crearon ni se modificaron nodos.

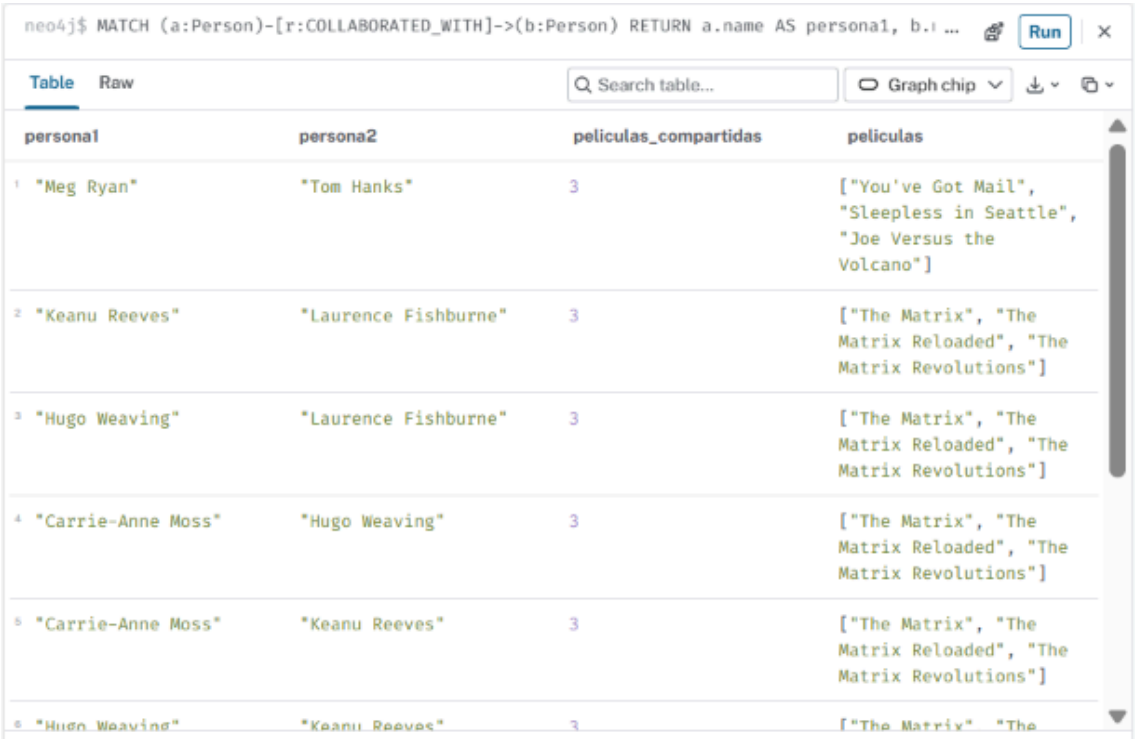
- Captura o tabla con el resultado obtenido:



- Comentario sobre la utilidad de la consulta en un sistema real: En un sistema real esto sirve porque evita tener que recalcular patrones costosos cada vez que se consulta. Es una técnica común de desnormalización en bases de grafos para acelerar consultas frecuentes.

Q17

- Propósito de la consulta: Lista las diez colaboraciones más fuertes del grafo, o sea los pares de actores que compartieron la mayor cantidad de películas usando la relación derivada creada en Q16.
- Explicación breve del patrón de grafo utilizado: Es un recorrido de un solo salto sobre la relación derivada, (a)-[:COLLABORATED_WITH]->(b). Ya no hay que pasar por los nodos Movie ni agrupar. movies y count se leen directamente de las propiedades de la relación y solo se ordena por count de forma descendente con LIMIT 10.
- Captura o tabla con el resultado obtenido:

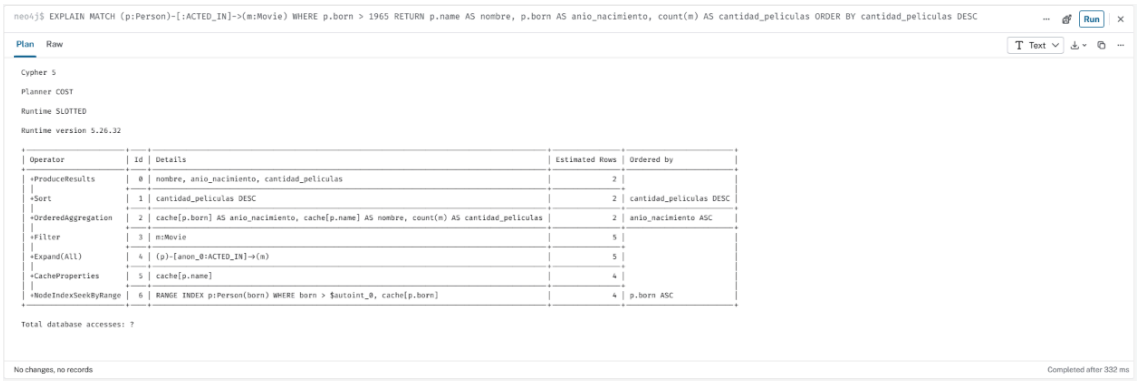


- Comentario sobre la utilidad de la consulta en un sistema real: Esta consulta permite responder en una sola lectura quiénes trabajan juntos con más frecuencia, sin volver a calcular el patrón costoso de Q16. Es útil para mostrar colaboradores frecuentes en el perfil de una persona, detectar equipos estables o alimentar un recomendador de contactos.

Q18

- Propósito de la consulta: Analizar cómo ejecuta Neo4j dos consultas de distinta complejidad: Q4 (dificultad media, con filtro y agregación) y Q9 (difícil, con ruta de profundidad variable). EXPLAIN muestra el plan estimado sin ejecutar la consulta, y PROFILE la ejecuta y reporta las filas y los accesos a base de datos (db hits) reales.
- Explicación breve del patrón de grafo utilizado: Esta es la comparación de los patrones ya vistos en Q4 que es un recorrido de un salto con filtro por propiedad y agregación y Q9 que es un shortestPath de profundidad variable entre dos nodos identificados por su nombre.
- Captura o tabla con el resultado obtenido:

EXPLAIN Q4



```
neo4j$ EXPLAIN MATCH (p:Person)-[:ACTED_IN]->(m:Movie) WHERE p.born > 1965 RETURN p.name AS nombre, p.born AS anio_nacimiento, count(m) AS cantidad_peliculas ORDER BY cantidad_peliculas DESC
```

Plan Raw

Text

Cypher 5

Planner COST

Runtime SLOTTED

Runtime version 5.26.32

Operator	Id	Details	Estimated Rows	Ordered by
+ProduceResults	0	nombre, anio_nacimiento, cantidad_peliculas	2	
+Sort	1	cantidad_peliculas DESC	2	cantidad_peliculas DESC
+OrderedAggregation	2	cache[p.born] AS anio_nacimiento, cache[p.name] AS nombre, count(m) AS cantidad_peliculas	2	anio_nacimiento ASC
+Filter	3	m:Movie	5	
+Expand(All)	4	(p)-[anon_0:ACTED_IN]->(m)	5	
+CacheProperties	5	cache[p.name]	4	
+NodeIndexSeekByRange	6	RANGE INDEX p:Person(born) WHERE born > $autoint_0, cache[p.born]	4	p.born ASC

Total database accesses: ?

No changes, no records

Completed after 332 ms

PROFILE Q4



```
1 PROFILE
2 MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
3 WHERE p.born > 1965
4 RETURN p.name AS nombre, p.born AS anio_nacimiento, count(m) AS cantidad_peliculas
5 ORDER BY cantidad_peliculas DESC
```

Plan Table Raw

Cypher 5

Planner COST

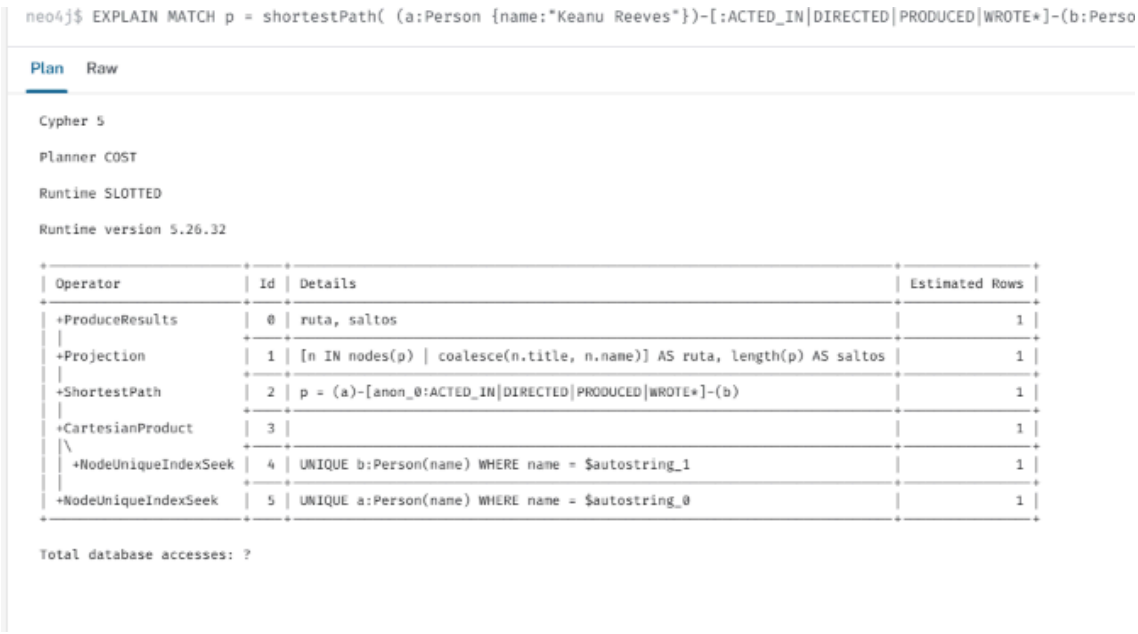
Runtime SLOTTED

Runtime version 5.26.32

Operator	Id	Details	Estimated Rows	Rows	DB Hits	Memory (Bytes)	Page Cache Hits/Misses	Ordered by
+ProduceResults	0	nombre, anio_nacimiento, cantidad_peliculas	2	39	0	0	0/0	
+Sort	1	cantidad_peliculas DESC	2	39	0	9008	0/0	cantidad_peliculas DESC
+OrderedAggregation	2	cache[p.born] AS anio_nacimiento, cache[p.name] AS nombre, count(m) AS cantidad_peliculas	2	39	0	1112	0/0	anio_nacimiento ASC
+Filter	3	m:Movie	5	54	54		0/0	
+Expand(All)	4	(p)-[anon_0:ACTED_IN]->(m)	5	54	366		0/0	
+CacheProperties	5	cache[p.name]	4	41	41		0/0	
+NodeIndexSeekByRange	6	RANGE INDEX p:Person(born) WHERE born > $autoint_0, cache[p.born]	4	41	42		0/0	p.born ASC

Total database accesses: 503, total allocated memory: 9440

EXPLAIN Q9



```
neo4j$ EXPLAIN MATCH p = shortestPath( (a:Person {name:"Keanu Reeves"})-[:ACTED_IN|DIRECTED|PRODUCED|WROTE*]-(b:Perso

Plan  Raw

Cypher 5

Planner COST

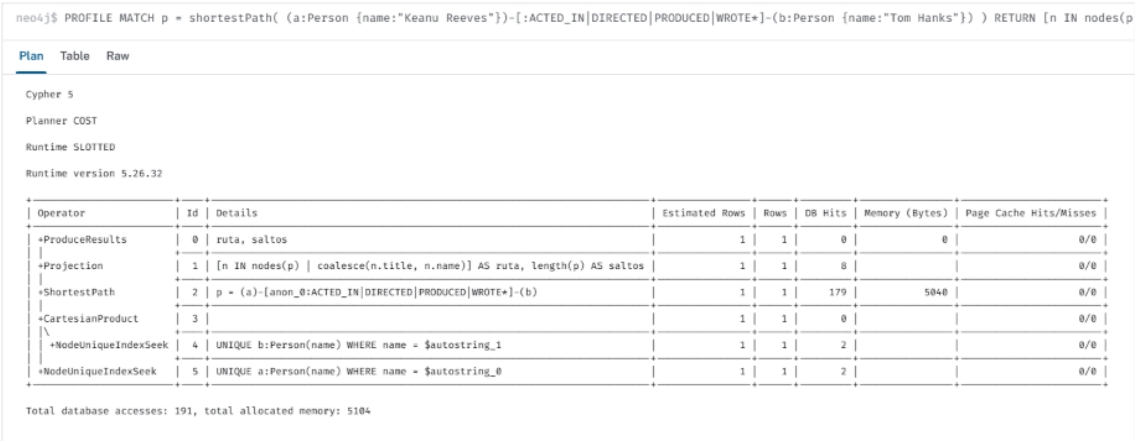
Runtime SLOTTED

Runtime version 5.26.32

+-----------------------+----+-------------------------------------------------------------------------+----------------+
| Operator              | Id | Details                                                                 | Estimated Rows |
+-----------------------+----+-------------------------------------------------------------------------+----------------+
| +ProduceResults       |  0 | ruta, saltos                                                            |              1 |
| |                     +----+-------------------------------------------------------------------------+----------------+
| +Projection           |  1 | [n IN nodes(p) | coalesce(n.title, n.name)] AS ruta, length(p) AS saltos |              1 |
| |                     +----+-------------------------------------------------------------------------+----------------+
| +ShortestPath         |  2 | p = (a)-[anon_0:ACTED_IN|DIRECTED|PRODUCED|WROTE*]-(b)                  |              1 |
| |                     +----+-------------------------------------------------------------------------+----------------+
| +CartesianProduct     |  3 |                                                                         |              1 |
| |\                    +----+-------------------------------------------------------------------------+----------------+
| | +NodeUniqueIndexSeek|  4 | UNIQUE b:Person(name) WHERE name = $autostring_1                         |              1 |
| |                     +----+-------------------------------------------------------------------------+----------------+
| +NodeUniqueIndexSeek  |  5 | UNIQUE a:Person(name) WHERE name = $autostring_0                         |              1 |
+-----------------------+----+-------------------------------------------------------------------------+----------------+

Total database accesses: ?
```

PROFILE Q9



```
neo4j$ PROFILE MATCH p = shortestPath( (a:Person {name:"Keanu Reeves"})-[:ACTED_IN|DIRECTED|PRODUCED|WROTE*]-(b:Person {name:"Tom Hanks"}) ) RETURN [n IN nodes(p

Plan  Table  Raw

Cypher 5

Planner COST

Runtime SLOTTED

Runtime version 5.26.32

+-----------------------+----+-------------------------------------------------------------------------+----------------+------+---------+-----------------+------------------------+
| Operator              | Id | Details                                                                 | Estimated Rows | Rows | DB Hits | Memory (Bytes)  | Page Cache Hits/Misses |
+-----------------------+----+-------------------------------------------------------------------------+----------------+------+---------+-----------------+------------------------+
| +ProduceResults       |  0 | ruta, saltos                                                            |              1 |    1 |       0 |               0 |                    0/0 |
| |                     +----+-------------------------------------------------------------------------+----------------+------+---------+-----------------+------------------------+
| +Projection           |  1 | [n IN nodes(p) | coalesce(n.title, n.name)] AS ruta, length(p) AS saltos |              1 |    1 |       8 |                 |                    0/0 |
| |                     +----+-------------------------------------------------------------------------+----------------+------+---------+-----------------+------------------------+
| +ShortestPath         |  2 | p = (a)-[anon_0:ACTED_IN|DIRECTED|PRODUCED|WROTE*]-(b)                  |              1 |    1 |     179 |            5040 |                    0/0 |
| |                     +----+-------------------------------------------------------------------------+----------------+------+---------+-----------------+------------------------+
| +CartesianProduct     |  3 |                                                                         |              1 |    1 |       0 |                 |                    0/0 |
| |\                    +----+-------------------------------------------------------------------------+----------------+------+---------+-----------------+------------------------+
| | +NodeUniqueIndexSeek|  4 | UNIQUE b:Person(name) WHERE name = $autostring_1                         |              1 |    1 |       2 |                 |                    0/0 |
| |                     +----+-------------------------------------------------------------------------+----------------+------+---------+-----------------+------------------------+
| +NodeUniqueIndexSeek  |  5 | UNIQUE a:Person(name) WHERE name = $autostring_0                         |              1 |    1 |       2 |                 |                    0/0 |
+-----------------------+----+-------------------------------------------------------------------------+----------------+------+---------+-----------------+------------------------+

Total database accesses: 191, total allocated memory: 5104
```

- Comentario sobre la utilidad de la consulta en un sistema real: Esta consulta es útil en un sistema real para analizar el rendimiento de las consultas antes de utilizarlas en producción. EXPLAIN permite conocer el plan que Neo4j pretende ejecutar sin realizar la consulta, mientras que PROFILE permite observar su ejecución real y detectar qué operaciones generan más accesos a la base de datos. Esto ayuda a identificar consultas costosas y decidir si es necesario optimizarlas mediante índices, cambios en los recorridos o mejoras en el modelo del grafo.

- En Q4, entre los principales operadores del plan aparecen NodeIndexSeekByRange, Expand(All), agregación y ordenamiento. El índice permite localizar las personas nacidas después de 1965 sin recorrer todos los nodos Person, mientras que Expand(All) recorre las relaciones ACTED_IN hacia las películas. Como mejora, sería conveniente mantener un índice sobre Person.born, ya que este tipo de filtro puede aparecer frecuentemente en consultas similares.
- En Q9, los operadores principales incluyen NodeUniqueIndexSeek, ShortestPath, Projection y ProduceResults. Los índices únicos permiten encontrar directamente a Keanu Reeves y Tom Hanks por nombre, mientras que ShortestPath realiza el recorrido entre ambos nodos. Una mejora posible sería limitar la profundidad del recorrido cuando el sistema conozca una distancia máxima razonable o utilizar relaciones derivadas como COLLABORATED_WITH para consultas de colaboración que se ejecuten con frecuencia.

4. Análisis de las consultas más complejas.

Las consultas de mayor dificultad de la práctica muestran una de las principales ventajas de una base de datos orientada a grafos que es básicamente posible expresar relaciones complejas mediante patrones relativamente directos en Cypher. A medida que las consultas avanzan, dejan de trabajar únicamente con nodos individuales y comienzan a utilizar recorridos de profundidad variable, agregaciones, exclusiones, caminos mínimos y relaciones derivadas.

Q9 y Q12 nos llamó más la atención por trabajar con rutas dentro del grafo. En Q9, shortestPath() permite encontrar la conexión más corta entre Keanu Reeves y Tom Hanks utilizando distintos tipos de participación en películas. Q12 utiliza un concepto similar para calcular el número de Bacon mediante relaciones ACTED_IN. Este tipo de consultas aprovecha directamente la estructura del grafo, aunque su costo puede aumentar conforme crece la cantidad de nodos y relaciones que deben explorarse.

Q10 y Q15 muestran un poco cómo el modelo también puede utilizarse para obtener conocimiento a partir de conexiones existentes. Q10 genera recomendaciones siguiendo la cadena FOLLOWS y REVIEWED, además de excluir películas que la persona ya reseñó. Por otra parte, Q15 compara películas mediante los actores que comparten y utiliza esa cantidad

como un puntaje de similitud. Ambos casos son ejemplos de consultas que podrían formar parte de sistemas reales de recomendación.

Otro aspecto importante aparece en Q16 y Q17. Q16 toma información que ya podía deducirse recorriendo actores y películas y la materializa mediante la relación COLLABORATED_WITH. Esto modifica el grafo para almacenar directamente una relación que antes debía calcularse. Q17 demuestra el beneficio de esa decisión, ya que las colaboraciones pueden consultarse posteriormente mediante un recorrido de un solo salto, sin volver a atravesar los nodos Movie ni repetir la agregación. Esto representa una forma de desnormalización orientada a mejorar consultas que se ejecutan frecuentemente.

Finalmente, el análisis con EXPLAIN y PROFILE demuestra que la dificultad lógica de una consulta no determina por sí sola su costo real. Q4 produjo 39 resultados y realizó 503 accesos a la base de datos, mientras que Q9 devolvió una única ruta y realizó 191 accesos. Los índices utilizados para localizar los nodos iniciales, la cantidad de relaciones recorridas y las operaciones posteriores como agregación u ordenamiento tienen un efecto directo sobre el rendimiento.

5. Conclusiones técnicas.

La práctica nos permitió aplicar los conceptos principales del modelo de grafos utilizando Neo4j y Cypher. A diferencia de un modelo relacional tradicional, las relaciones forman parte directa de la estructura de los datos, por lo que consultas relacionadas con colaboración, seguimiento, recomendaciones y rutas pueden expresarse recorriendo conexiones entre nodos.

Durante las consultas se utilizaron diferentes recursos de Cypher como filtros con WHERE, agregaciones con count() y collect(), relaciones opcionales mediante OPTIONAL MATCH, exclusiones con NOT EXISTS, recorridos de profundidad variable y búsqueda de rutas mediante shortestPath(). También se comprobó que las propiedades pueden almacenarse tanto en nodos como en relaciones, como ocurre con roles en ACTED_IN y rating en REVIEWED.

La creación de COLLABORATED_WITH permitió comprobar cómo un grafo puede enriquecerse con conocimiento derivado. Al almacenar directamente una colaboración que anteriormente debía calcularse a través de las películas, consultas posteriores pueden realizar

recorridos más simples. Esta estrategia puede mejorar el rendimiento cuando determinada información derivada se consulta frecuentemente, aunque implica mantenerla actualizada cuando cambian los datos originales.

Finalmente, EXPLAIN y PROFILE permitieron observar que el rendimiento depende tanto del patrón de consulta como de los índices disponibles y de la cantidad de relaciones procesadas. El uso de índices para localizar nodos y el diseño adecuado de los recorridos son elementos importantes al trabajar con grafos de mayor tamaño. En general, Neo4j resulta especialmente adecuado para problemas donde las conexiones entre entidades son tan importantes como las propiedades de las propias entidades.